How a Machine Becomes Trusted

ADTS device registration and authentication: the full process

This is the whole lifecycle told as a sequence of events: what each person and system does, what the server records, and why each step exists. It covers everything from a PC arriving at a site to it making its thousandth API call, and what happens when trust is withdrawn.

The purpose

The server should answer API requests from one kind of caller only: a physical PC that the organisation deliberately registered, that a human approved, and that can still prove it is that same PC. A password can be shared, a file can be copied, and a token can be stolen, so none of those is enough on its own. The design therefore rests on a key that is created inside the machine's TPM chip and can never leave it. Every other part of the process exists either to connect that key to an authorised machine, or to check on every request that the key is still present.

Before anything happens: preparing the ground

Before any PC is touched, three things are set up.

The inventory. The organisation records every PC it expects to deploy in a machine inventory: its hardware ID, the site it belongs to, and the public internet address range that site's traffic comes from. This is the list of machines that are allowed to exist. A PC not on this list can't even be issued a token.

The admin accounts. Admin accounts are created in the portal and protected with multi-factor authentication. Each admin gets one or both of two roles. A token issuer can start a registration. A device approver can complete one. The system later makes sure the same person never does both for the same machine.

The server's identity. The server gets a signing key held in a key management service, so the private key never exists as a file. That key is vouched for by a root key kept offline. The root's public half is built into the client application. From the first time the client runs, it already knows how to recognise the real server, and it never has to trust a key the server simply hands it.

Stage one: an admin issues a registration token

A PC has been installed at a site, and someone needs to register it. An issuing admin logs into the portal, passes MFA, and requests a token for that machine's ID.

The server works through its checks in order. Is the session genuine and MFA-verified? Does this admin hold the issuer role? Is the request carrying a valid anti-forgery token? Is the admin within their rate limit? Then it looks at the machine itself. Is it in the inventory, and still active? Does it already have a device registered? If it does, issuing another token would be pointless and possibly harmful, so the request is refused. Is there already an unused token outstanding for this machine? If so, the request is refused unless the admin explicitly asks to replace it. In that case the old token is revoked first and the revocation is recorded.

Once everything passes, the server generates a random token: 125 bits, written in an alphabet that leaves out easily confused characters. It does **not** store the token. It stores a keyed hash of it, together with the machine ID, the site's permitted network range (taken from the inventory, never from what the admin typed), the identity of the issuing admin, and an expiry fifteen minutes away. An audit entry records that a token was issued, by whom, for which machine, from which address, but not the token itself.

The portal shows the token exactly once. After that nobody can retrieve it again, not even from the database, because only its hash exists there. The timing is strict because the database enforces it: if two admins click at the same instant, only one token is created.

Stage two: handing over the token

The admin reads the token to the person at the machine, by phone or another agreed secure channel. It is never emailed, pasted into chat, or written into a ticket. This step involves no software, but it matters: anyone who intercepts the token during these fifteen minutes holds the one thing that can start a registration.

Stage three: the machine registers itself

The on-site person runs the registration command in the ADTS client. Because it's an administrative action, the service accepts it only from an approved admin account on that PC. They type in the token. The client keeps it in memory only and never writes it to disk.

Before contacting the server, the client checks that the machine has a working TPM. If it doesn't, registration stops there. There is no fallback to a software key, because a software key can be copied and would undermine everything that follows.

The client then asks the TPM to generate a key pair. The private half is created inside the chip, marked non-exportable, and tied to the machine rather than to any user account. Only the public half can be read out. The client computes a short fingerprint of that public key and uses the private key to sign a message containing the token, the machine ID, the public key and the hostname. This signature proves the machine actually holds the private key matching the public key it is presenting.

The client sends everything to the server: token, machine ID, hostname, the IP addresses it can see locally, the public key and the signature.

The server checks the cheap, purely mathematical things first, before touching the database. Is the request well-formed? Is the public key valid? Does the signature verify? Then it checks the token. Does its hash match a record? Is it still unused and unexpired? Does the machine ID match the one the token was issued for? Is the request coming from inside that site's permitted network range?

That last pair of checks deserves a closer look. If a valid token arrives with the **wrong** machine ID or from the **wrong** network, something is badly wrong: the token has leaked, or someone is trying to use it on another machine. The server doesn't just reject the request. It revokes the token immediately, records the misuse and raises an alert. A token that has been seen in the wrong place is never given a second chance.

If all checks pass, the server makes three changes in a single transaction. It marks the token consumed, creates a device record with status pending approval (storing the public key, fingerprint, hostname, enrollment address and reported IPs), and links the token to that device. It writes two audit entries: token consumed, device enrolled. If any part fails, none of it takes effect.

The client receives the device ID and the key fingerprint, and shows the fingerprint on screen.

At this point the token's job is finished for good. It can never be used again, and nothing later in the process depends on it.

About the key fingerprint

The key fingerprint is a short code that stands in for the device's public key, so two people can compare that key over the phone. The full public key is too long to read aloud, so both the client and the server hash it with SHA-256 and keep the first 20 hex characters, grouped in fours (for example 3F9A-C21E-77D4-0B58-E6A1). The same key always gives the same fingerprint; a different key gives a completely different one.

The fingerprint is not a secret and cannot authenticate anything by itself. It exists so that, during approval, a human can confirm that the key the server stored is the key sitting in the real machine's TPM.

Stage four: a second person approves

A device that is only pending has no access to anything. It exists as a record waiting for a decision.

A **different** admin, one holding the approver role, opens the pending device in the portal. They see the machine ID, hostname, where the enrollment came from, and who issued the token. Before approving, they phone the on-site person, who reads out the fingerprint displayed by the client. The approver types it in.

This step is the reason the process can't be defeated by someone swapping in their own key. If an attacker had somehow enrolled a key of their own, the fingerprint on the real machine's screen would not match the one in the database, and the approval would fail.

The server confirms the approver passed a fresh MFA check within the last few minutes, is not the person who issued the token, typed a fingerprint that matches exactly, and is acting on a device that is still pending and less than a day old. It then moves the device from pending approval to active with a conditional update. If someone else changed the device in the meantime, the update affects no rows and the approval is refused rather than silently succeeding. The approval is recorded, including who approved.

If the approver rejects the device instead, or nobody acts within 24 hours, the device is marked rejected. That is final. The PC needs a new token and has to start over.

Stage five: the machine waits and checks in

While waiting, the client periodically asks the server whether it has been approved. It doesn't just send its device ID and read back a status, because that would let anyone probe which devices

exist. Instead it asks for a one-time challenge: a random value the server generates, stores, and allows sixty seconds to use.

The client signs that challenge with its TPM key and sends the signature back. The server marks the challenge used immediately, even before checking the signature, so no challenge can be tried twice. It then verifies the signature against the public key **stored at enrollment**, never against a key supplied in the request. Only after that does it reveal the status.

If someone asks for a challenge for a device ID that doesn't exist, the server still returns something that looks exactly like a real challenge. It simply can never succeed. From outside, real and non-existent devices look the same.

The client keeps polling, gradually waiting longer between checks. When the answer is active, it moves on. When the answer is rejected or revoked, it stops and tells the operator a new registration is required.

Stage six: the machine opens a session

An active device still doesn't send its TPM signature with every API call. TPM operations are slow, and a machine making many requests would be held back. Instead it proves its identity once, to open a session that lasts two hours.

It requests a fresh challenge, this time for session use. It also generates a temporary key pair in ordinary memory, used for this session only. It signs a message tying together its device ID, the challenge and that temporary public key, and sends it. The request contains no public key for the device and no enrollment token. The server already knows who this device should be.

The server uses up the challenge, confirms the device is still active, checks the signature against the stored key and, where the site requires it, checks the request comes from the site's network. Then it generates its own temporary key. Using both temporary keys, client and server each independently compute the same shared secret, without that secret ever being sent. From it they derive two session keys: one for proving each request is authentic and unaltered, and one for encrypting request and response bodies.

The server creates a session record. It stores a hash of a newly generated access token, the session keys encrypted under the key management service, the source address and a two-hour expiry. Any older session for the same device is shortened to expire within five minutes, so there's no gap during the handover but only one session remains in the long run. The server signs its response with its protected signing key and records that a session was issued.

The client doesn't take the response at face value. It checks that the server's signing key is vouched for by the root key built into the application, and that the response was signed by it. If either check fails, the client throws the whole response away. This is how the client knows it is talking to the real server and not something in between. Only then does it derive the same session keys and keep them in memory, never on disk.

Stage seven: everyday work

Now the machine can fetch its app configuration, hardware configuration and everything else it needs.

Each request carries the access token, a timestamp, a unique request ID, and a code computed with the session key over the method, path, timestamp, request ID and a hash of the body. The body itself is encrypted.

The server behaves like a strict gatekeeper. Is there a session for this token, not expired and not revoked? Is the device behind it still active? Is the timestamp within a minute of the server's clock? Does the code match what the server computes with its copy of the session key? Has this request ID been used before in this session? If so, it's a replay and is refused. Does the thing being requested actually belong to this machine? One PC must never be able to read another PC's configuration just by changing an ID in the request.

Only when every check passes does the request reach business logic. When any check fails, the client gets a flat refusal with no explanation, and the real reason goes to the audit log. Those denial entries are rate-limited so a flood of bad requests can't bury the log.

The design also closes off a very common mistake. Protection doesn't depend on each developer remembering to add it to each new endpoint. Every route is protected by default, only the handful of registration endpoints are exempt, and an automated test fails the build if a route slips through unprotected.

Stage eight: renewal

A little before the two hours are up, the client quietly goes through stage six again: a new challenge, a new signature, a new session. The previous session lingers for a few minutes so nothing in progress is cut off, then it expires. The machine repeats this indefinitely, and every renewal is another moment where the server re-checks that the device is still active and still holds its key.

When trust is withdrawn

An admin can **suspend** a device, temporarily, for example while investigating something. They can **reinstate** it later, or **revoke** it permanently. Suspension and revocation both require a fresh MFA check. They change the device's status and, in the same transaction, revoke every open session. The change reaches running servers within seconds, so the machine's very next request is refused. It doesn't have to wait for its session to expire.

A revoked device stays revoked. If a PC is reimaged or replaced, its TPM key is lost with the old installation, so it starts again from stage one with a new token and a new approval. The old record remains in the database as history.

Housekeeping in the background

A few scheduled jobs keep things tidy without people having to remember. Tokens nobody used are marked expired once their fifteen minutes run out. Devices left pending for more than a day are rejected. Used challenges and request IDs are cleared once they're old enough that they could never be valid again. Old session records are removed after a week.

The audit log is never cleared. Every night its integrity is checked: each entry carries a hash of itself linked to the previous entry, so any alteration or deletion breaks the chain and is detected. The latest link is also copied to storage that can't be modified.

The record left behind

Every event across the lifecycle leaves an entry in that audit log: tokens issued, replaced, used, expired or revoked; enrollments accepted or refused; approvals and rejections; suspensions and revocations; sessions opened and denied; requests refused. Each entry says who or what acted, when, from where, what the state was before and after, and, for refusals, exactly why.

The database role the application uses can add entries but can't change or delete them, and the hash chain makes any tampering evident. For any machine you can reconstruct its full history: when its token was issued and by whom, when it enrolled, how long it waited for approval, who approved it, every session it opened, and when and why it was stopped. The same record is what you would show an auditor to prove the system enforced its own rules.

Summary

A machine can only get into the system if an admin sends it an invitation tied to that exact hardware and network. That invitation can be used once, for fifteen minutes. It only counts if the machine proves it holds a key that can't be copied, and it only becomes real when a second person confirms, by voice, that the key is the right one. From then on the machine proves itself again every two hours and signs every single request. The server refuses anything that doesn't check out, remembers everything that happened, and can shut a machine out within seconds.